Issue #64
Revised contract implementation

LiDRON Autonomous Systems | October 18, 2026

Change report and partner testing instructions

Aligned the execution runner with Appendix A of Gazebo_Scenario_Updated.pdf for issue #62. This revision replaces the previous change report and test expectations. It stays on the existing codex/issue-64-scenario-runner branch.

Changes achieved

- Added the canonical python -m scenario_framework run command and shared Runner, Monitor, Validator and ResultStore interfaces. Existing script commands remain aliases.

- Added Draft 2020-12 JSON Schemas for scenarios, events and results, with semantic configuration checks.

- Standardized execution_status, separate PASS/FAIL verdicts, reasons, phases, cleanup precedence and CI exit codes.

- Recorded state, full paths, valid NED position, obstacles, service responses and process exits in one sequenced event stream, retaining source clock information.

- Added both repository commit IDs, configuration hash, UTC start, evidence checks and artifact paths to result output. Provisional evidence is saved before cleanup and final output is atomic.

- Added monitoring-loss handling, host deadline supervision, dirty-stack refusal and an explicit full-teardown reset.

- Updated the Python tests, README and this partner testing guide.

Verification and acceptance boundary

All 26 new local tests pass using fake processes and ROS doubles, including JSON Schema validation of recorded evidence. Real Gazebo/PX4 flight, route clearance, spawn/reset and repeated lab execution remain unverified. The existing ROS/perception pytest suite was not run here.

No automatic PASS is claimed. Detailed required-state/path/arrival and causal evaluation belongs to #65. An otherwise normal COMPLETE run currently reports FAIL/ERROR/MONITOR_DATA_MISSING at validation, while execution.json retains the observed COMPLETED execution. Known forbidden states can produce COMPLETED/FAIL/EXPECTATION_FAILED. V2 obstacle injection and replanning checks remain in #65/#66.

Branch: `codex/issue-64-scenario-runner`. Updated October 18, 2026 against `Gazebo_Scenario_Updated.pdf`, especially Appendix A. This guide replaces the previous testing instructions. Scope is v1 execution and shared interfaces.

What changed

The canonical command is now `python -m scenario_framework run <scenario.json> --output results`. The old `scripts/run_scenario.py` command remains an alias. The package provides Runner, Monitor, Validator and ResultStore interfaces. The runner owns processes, monotonic startup/mission/cleanup deadlines, cancellation, and full teardown. The monitor attaches before destination submission and enable.

The v1 scenario still selects the default Simulation world/spawn, no injected obstacles, and local NED destination (north=8, east=0, down=-5 m). A Draft 2020-12 JSON Schema validates structure; semantic checks reject nonfinite values and contradictory required/forbidden states. Unsupported v2 trigger/obstacle fields are rejected. This does not add obstacle injection or v2 replanning validation.

Readiness requires perception/mission/diagnostics nodes, fresh IDLE, valid PX4 position and obstacle messages, all mission services, discovery of the path publisher, and successful `/autonomy/preflight_check`. That check verifies camera, LiDAR, GPS and PX4 input health. A path message is not required before mission start. Destination must return accepted=true; enable must return success=true. Required topic/telemetry loss during a mission becomes ERROR/MONITOR_DATA_MISSING.

Cleanup is always attempted after launch, including partial startup, timeout and cancellation. A per-user lock prevents concurrent runners. An already running manual stack is refused without stopping it. A dirty-stack latch is set before launch and cleared only after successful teardown. Cleanup failure or an abruptly killed runner blocks further reuse until the explicit reset command succeeds.

Important: execution completion does not imply PASS

The new contract separates `verdict` (PASS/FAIL) from `execution_status` (COMPLETED/TIMED_OUT/ERROR/CANCELLED). No `FAILED` status exists. Observing HOLD or ABORT is recorded as a state event, followed by a COMPLETED execution with an expectation-related FAIL when that forbidden state is observed. The evidence sequence number identifies the offending event.

This #64 implementation includes the conservative validator interface and the forbidden-state check. Detailed required-state sequence, trajectory/arrival and cross-topic causal evaluation belong to #65 and remain unimplemented. Therefore a normal run reaching COMPLETE currently ends with verdict=FAIL, execution_status=ERROR, reason=MONITOR_DATA_MISSING, phase=validation. Its `execution.json` still records the provisional COMPLETED execution. This is the expected unverified result for this branch, not a passing flight test. A PASS must wait for the detailed validator; receipt order is never treated as proof of causality across ROS and PX4 clock domains.

Situation	verdict	execution_status / reason	Exit
Fully validated mission and cleanup (future #65)	PASS	COMPLETED / NONE	0
Observed forbidden HOLD/ABORT	FAIL	COMPLETED / EXPECTATION_FAILED	1
Mission deadline expired	FAIL	TIMED_OUT / TIMEOUT	124
Startup deadline expired	FAIL	ERROR / TIMEOUT	2
Missing monitoring or detailed validation	FAIL	ERROR / MONITOR_DATA_MISSING	2
Bad configuration	FAIL	ERROR / CONFIG_INVALID	2
Service/process/infrastructure fault	FAIL	ERROR / INFRASTRUCTURE_ERROR	2
Ctrl-C or SIGTERM	FAIL	CANCELLED / CANCELLED	130
Cleanup failed, regardless of prior result	FAIL	ERROR / CLEANUP_ERROR	2

Cleanup failure sets phase=cleanup and preserves the previous status, verdict, reason and phase in `original_outcome`, plus all prior causes in `reasons`. Successful cleanup does not erase a prior failure. The teardown subprocess is bounded by cleanup_s; stopping a process group may add up to two seconds.

Result and event files

Each unique run folder under the selected output directory contains:

- `scenario.json`: resolved scenario snapshot.

- `manifest.json`: run ID, UTC start, both repository commit IDs, canonical JSON

configuration SHA-256 and monotonic run origin. Unknown source commits prevent launch and are explicitly null in error reports rather than fabricated.

- `events.jsonl`: one shared event stream. Each event has schema_version=1,

run_id, strictly increasing seq, elapsed_s from the common run origin, ros_time_s (or null), source, kind and payload.

- State events carry previous/current. Path events archive all points, path ID,

count, frame and source ROS stamp. Position events preserve local NED values, validity and PX4 boot timestamp with its explicit clock domain. Untimestamped String/Bool messages use ros_time_s=null. Obstacle events carry detected and optional distance. Service responses and process exits use Appendix A payloads.

- `execution.json`: the ROS adapter's provisional execution outcome.

- `provisional-result.json`: outcome saved before cleanup.

- `result.json`: atomic final outcome after cleanup, with schema_version, run_id,

scenario_name, verdict, execution_status, reason, phase, duration_s, cleanup_status, original_outcome, reasons, start_utc, source_commits, configuration_sha256, expectation_checks and artifact_paths.

- `stack.log`, `mission.log`, `cleanup.log`: process diagnostics; `progress.json`

shares the mission deadline with the host watchdog.

Cleanup status is COMPLETED, ERROR or NOT_REQUIRED. Each expectation check carries passed and evidence_seq; deferred checks are explicitly unsuccessful/unverified. Artifact paths are relative to the run folder. Provisional evidence survives a finalization error. Unwritable output is reported as infrastructure ERROR on stderr. Configuration rejection prints structured JSON on stderr before launch; it does not create a run folder. Output must stay inside this repository because Docker mounts it at `/autonomy`. Default `results/` is ignored by Git.

Setup and local checks

Share this branch with your partner before they fetch it; it is not automatically published. Use Ubuntu 24.04, Docker/Compose, a working Gazebo display, Python 3.10+ and matching Simulation checkout. On macOS run these commands inside the Ubuntu VM, as described in README; do not use host Docker Desktop for the flight tests. From the repository root:

```
git fetch origin
git switch codex/issue-64-scenario-runner
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-scenarios.txt
export SIMULATION_ROOT=/absolute/path/to/Simulation
./scripts/simulation.sh check
python -m unittest discover -s tests -v
```

Expected: 26 tests pass. These use fake processes/ROS doubles and validate produced events/results against JSON Schemas. They cover service order, publisher discovery, payload clocks, monitoring loss, timeouts (including a hung helper), cancellation, cleanup precedence, lock/refusal/dirty reset, invalid scenarios, atomic output failure and conservative verdicts. They do not prove flight success. The existing ROS/perception pytest suite has not been run in this environment.

Lab check 1: normal mission, evidence and fresh repeat

```
python -m scenario_framework run scenarios/clear_path.json --output results
runner_code=$?
echo "Runner exit: $runner_code"
./scripts/simulation.sh running
```

Expected: stack builds/starts, waits for readiness, sets destination and enables without manual commands. A successful preflight response precedes the accepted destination response, which precedes successful enable in events.jsonl. All subscriptions are already attached. No pre-mission path message is required.

Inspect default spawn, route clearance, takeoff, motion toward 8 m north at 5 m altitude, landing and COMPLETE. These behaviors remain unverified in the lab. For a first uncached build, copy the scenario to /tmp and raise startup_s to 900.

For an otherwise normal flight, this branch's expected final result is FAIL/ERROR/MONITOR_DATA_MISSING at validation, exit 2, cleanup_status=COMPLETED. The provisional execution.json should show COMPLETED and last_state=COMPLETE. No running container IDs should remain. Run it three times and verify a fresh world/spawn/IDLE and a unique evidence folder each time. Record every failed attempt too. Do not reinterpret these unverified outcomes as PASS.

Lab check 2: mission timeout and clean rerun

```
python - <<'PY'
import json
from pathlib import Path
p = json.loads(Path('scenarios/clear_path.json').read_text())
p['name'] = 'mission_timeout'
p['timeouts']['startup_s'] = 900
p['timeouts']['mission_s'] = 0.1
Path('/tmp/mission_timeout.json').write_text(json.dumps(p))
PY
python -m scenario_framework run /tmp/mission_timeout.json --output results
runner_code=$?
echo "Runner exit: $runner_code"
./scripts/simulation.sh running
```

Expected: FAIL/TIMED_OUT/TIMEOUT, phase=mission, exit 124 and bounded teardown with cleanup_status=COMPLETED. No running IDs remain. It may terminate at takeoff. Then rerun the normal scenario and confirm fresh spawn and state. Its final verdict still has the validation limitation described above.

Lab check 3: startup timeout, cancellation and monitoring loss

Startup timeout: use a temporary scenario with startup_s=0.1. Expected: FAIL/ERROR/TIMEOUT at startup, exit 2, no enable response. If launch began, cleanup_status=COMPLETED; if timeout happened before launch, NOT_REQUIRED is valid. For the controlled withheld-component test in Appendix B, stop one required component in a dedicated simulation and confirm no mission-enable call occurs.

Cancellation: run a normal scenario, press Ctrl-C once during startup or flight. Expected: FAIL/CANCELLED/CANCELLED, exit 130 and cleanup_status=COMPLETED. Wait for cleanup. A second interruption during teardown can produce CLEANUP_ERROR.

Monitoring loss: in a dedicated test run after enable, stop the Simulation `ros` service using the same Compose project/configuration used to launch it. This withholds the sensor bridge. Expected: within the health window (about two seconds plus service response time), FAIL/ERROR/MONITOR_DATA_MISSING at mission, exit 2 and cleanup. If the whole launcher exits first, INFRASTRUCTURE_ERROR is also valid; keep logs to distinguish process failure from lost telemetry.

Lab check 4: failure interface, refusal and configuration rejection

During mission execution, publish an intentional forbidden state in another terminal (simulation only):

```
docker exec lidron_autonomy bash -c '
 source /autonomy/install/setup.bash &&
 ros2 topic pub --once /autonomy/state std_msgs/msg/String "{data: HOLD}"
'
```

Expected if HOLD is received while monitoring is healthy: FAIL/COMPLETED/ EXPECTATION_FAILED, phase=validation, exit 1. The check must reference the HOLD event's seq. This injects a state message; it does not prove a real flight fault. Keep events because concurrent publisher timing can affect which state arrived.

Manual-stack refusal: start `./scripts/simulation.sh up` separately, then invoke the runner. Expected: FAIL/ERROR/INFRASTRUCTURE_ERROR and the existing stack untouched. Stop it manually before continuing. A second concurrent runner must also be refused without interfering with the first run.

Configuration rejection: change a temporary copy to mission.frame=enu, a negative timeout, nonfinite number or version=2/triggers fields. Expected: JSON diagnostic ERROR/CONFIG_INVALID at config on stderr, exit 2, no launch or teardown.

Lab check 5: cleanup error and recovery

The local fake-adapter test already injects shutdown failure. In a controlled lab fault-injection harness, make the shutdown adapter fail after an observed HOLD. Expected: FAIL/ERROR/CLEANUP_ERROR at cleanup, cleanup_status=ERROR, original_outcome=COMPLETED/FAIL/EXPECTATION_FAILED, and both reasons retained. Do not break the shared Docker daemon to simulate this test.

A subsequent normal runner invocation must refuse the dirty stack. Resolve the infrastructure fault, then perform an explicit full teardown:

```
python -m scenario_framework reset --cleanup-timeout 30
```

Expected: successful teardown, no running stack containers, dirty latch cleared. The next run launches from fresh state. A failed reset retains the latch.

Evidence and acceptance boundary

Send the complete run folder, scenario, console output and exit code, both Git commit IDs, OS/Docker versions and spawn/flight/landing screenshots or recording. For every check record expected vs actual, teardown time, leftover containers and whether the next run was fresh. Never discard failed attempts.

Appendix B's full acceptance matrix (three clear_path and three alternating obstacle_replanning runs with 6/6 PASS, expectation-failure checks and versioned scenario extension) depends on #65/#66/#67. V2 geometry/transforms, injection windows, replacement-path clearance/progress and fresh arrival evidence remain outside this #64 change. A republished path or state name alone cannot satisfy those checks. #64 contract/lifecycle testing can proceed now; complete framework validation and issue closure must retain these explicit dependencies.

No actual Gazebo/PX4 success is claimed. This runner tears down a simulation; it must not be used as a flight-control shutdown mechanism on physical hardware.